

Embedded Linux and Yocto Golden Gift

From 01/07/2026 to 28/08/2026

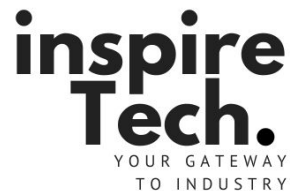
Every Monday, Wednesday and Friday (9PM - 11PM)

Zoom Meeting ID: 821 6580 1728, Passcode: 574596



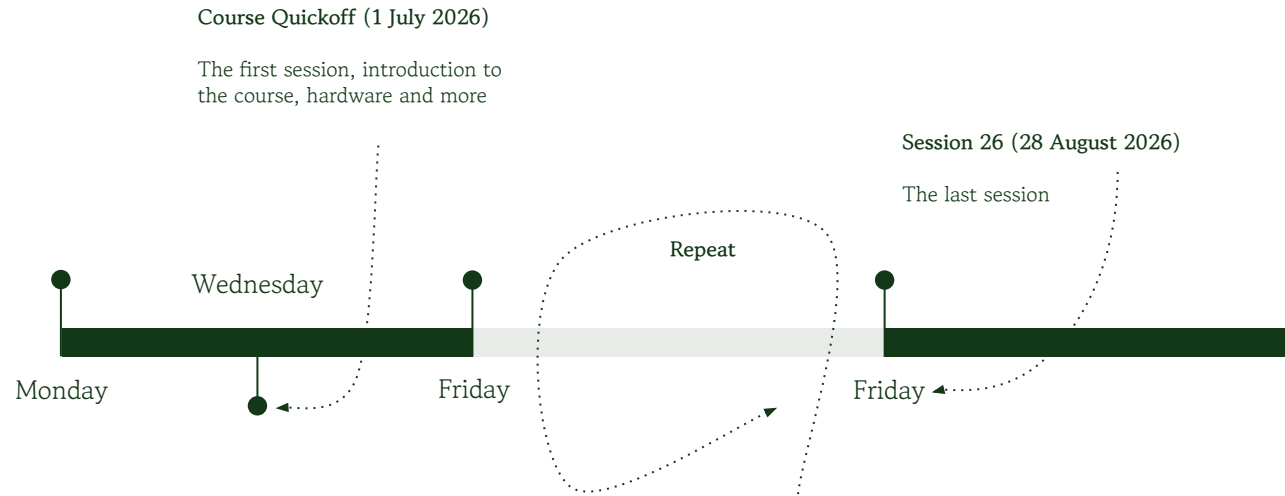
Talel BELHAJ SALEM

Yocto Ambassador ([linkedin.com/in/bhstalel](https://www.linkedin.com/in/bhstalel))



Special thanks to [InspireTech](#) for helping this course to be free

Special thanks to Faouzi GANNAR for helping with Zoom



Hardware

Raspberry Pi Zero 2W

Quad-core 64-bit Cortex-A53 at 1GHz

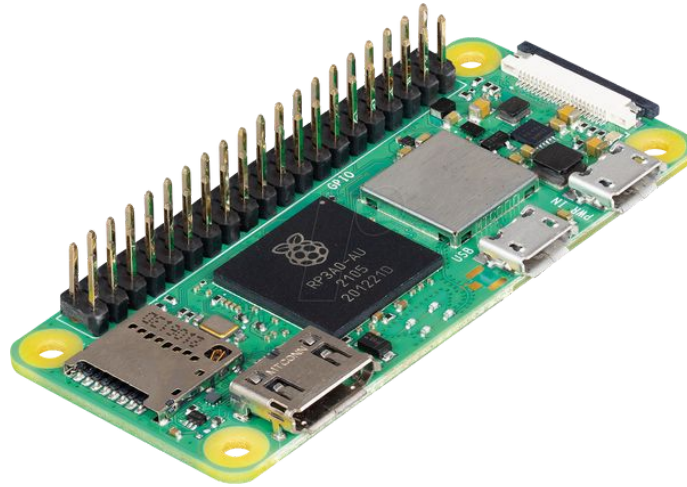
512MB RAM

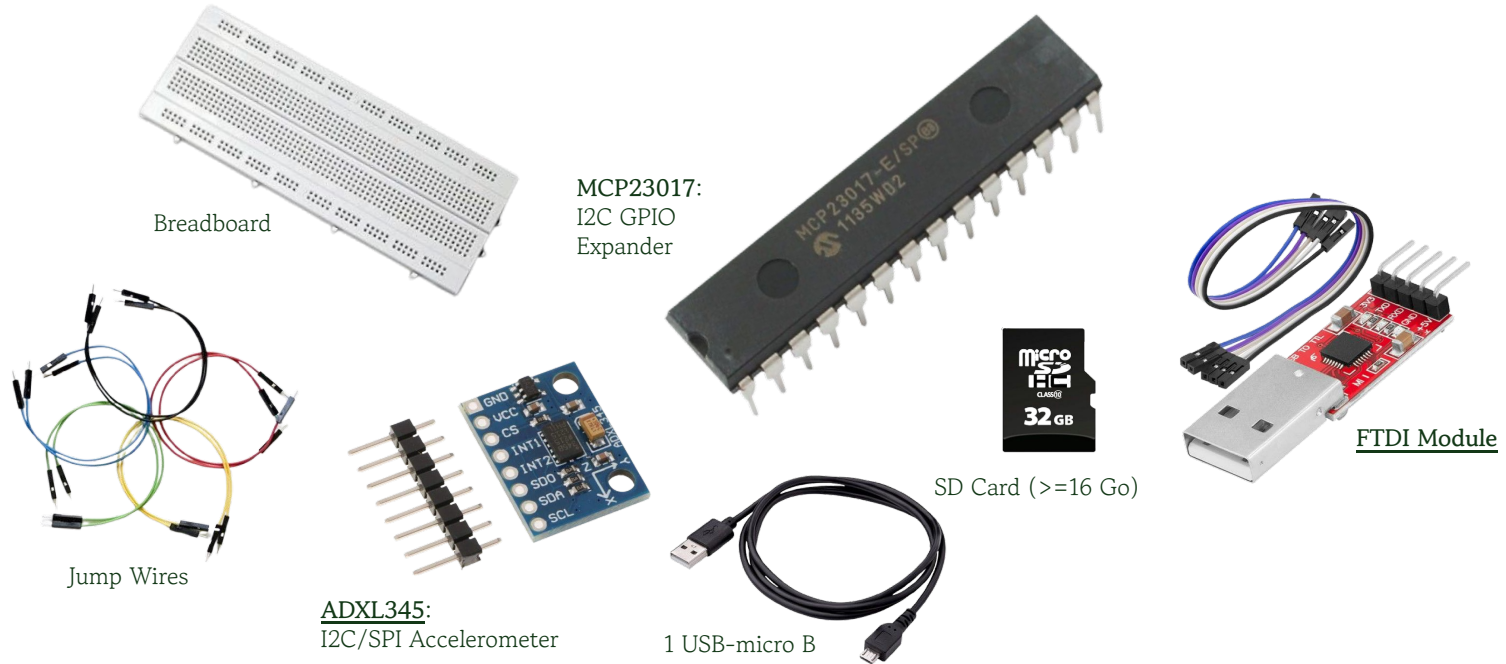
40-pin connector

WiFi 802.11b/n/g

Bluetooth 4.2, BLE

Mini HDMI





General Planning

Part 1	Part 2	Part 3
- Architecture - Shell Programming - POSIX Programming <i>Lab 1</i> Start developing shell scripts and POSIX programs cross compiled for the target board	- Toolchains - Libraries - Make and Cmake <i>Lab 2</i> Develop Linux-compliant shared and static libs, with pkgconfig support for the target board	- Kernel and U-boot - Modules and sysfs - GPIO, SPI and I²C <i>Lab 3</i> Develop kernel modules, device tree overlays and handle U-boot configuration

Part 4 (<i>Yocto Part I</i>)	Part 5 (<i>Yocto Part II</i>)	Part 6 (<i>Security & OTA</i>)
→ BitBake and Metadata → Recipes Development → Images and Distros <i>Lab 4</i> Prepare your first Yocto build, develop metadata and optimize Yocto distributions	→ BSP → Machine Development → Kernel and Initramfs <i>Lab 5</i> Design and develop custom Yocto BSP layer, initramfs, and handle kernel fragments	→ dm-crypt → dm-verity → RAUC <i>Lab 6</i> Encrypt partitions, check their integrity, integrate SELinux and an OTA update solution

Detailed Planning

❑ Architecture

- ❑ Understand the Linux architecture
- ❑ Understand the difference between **Userspace** and **Kernelspace** and components of each part
- ❑ Understand how the CPU participates in the separation between Userspace and Kernelspace and how they communicate

❑ Shell Programming

- ❑ Understand in details the **Rootfs** (*Root Filesystem*)
- ❑ Learn about the **Shell** interpreter
- ❑ Learn about Shell commands and **Permissions**
- ❑ Learn about various Shell features (*input/output redirection, piping, ...*) and programming (*if, while, variables, for, ...*)
- ❑ Learn about POSIX-compliant shell scripts

❑ POSIX Programming

- ❑ Learn about IPC (Inter Process Communication)
- ❑ Learn about file/signal handling and multi threading/processing

❑ Labs Summary (*to be communicated later*)

- ❑ Setup your first image and boot process for the target device
- ❑ Prepare an SDK for the target board
- ❑ Develop system management shell scripts
- ❑ Cross compile POSIX programs for the target board

❑ Toolchains

- ❑ Learn the compilation process (*from **preprocessing** to **linking***)
- ❑ Learn about Compiler and Linker flags
- ❑ Learn types of compilation (*native, cross, canadian, ...*)
- ❑ Learn how to create a Toolchain with the **crosstool-ng** project

❑ Libraries

- ❑ Learn about **static** and **dynamic** (*shared*) libraries
- ❑ Learn about linker/loader cache
- ❑ Simulate executables with **Qemu** project

❑ Make/Cmake/Pkgconfig

- ❑ Learn to write complete **Makefiles**
- ❑ Learn how to structure C and C++ projects
- ❑ Learn how to add support for Cross compilation to the project
- ❑ Learn about the **Cmake** and **Pkgconfig** projects

❑ Labs Summary (*to be communicated later*)

- ❑ Develop a library and produce a static and shared variants
- ❑ Develop a project that uses the library and cross compile it
- ❑ Deploy everything for the target board

❑ Kernel and U-boot

- ❑ Learn the structure of the source code of the Linux kernel, U-boot and various **subsystems**
- ❑ Learn how the Kernel is compiled and the output components
- ❑ Learn about the Kernel build system (**Kbuild**), Device Trees (**dts**) and Device Tree Overlays (**dtbo**)
- ❑ Learn how to enable/disable and integrate **symbols** in the Kernel

❑ Modules and sysfs

- ❑ Learn about Kernel modules code structure
- ❑ Learn about **sysfs** devices, drivers, classes, objects and more
- ❑ Develop **Character** device drivers

❑ GPIO, SPI and I²C

- ❑ Learn about the **SPI** and **I²C** kernel subsystems and how to integrate their drivers
- ❑ Learn how to manipulate SPI and I²C from userspace
- ❑ Learn about how to control GPIO from userspace manually via sysfs
- ❑ Learn about how to control GPIO from userspace with **libgpiod**

❑ Lab Summary (*to be communicated later*)

- ❑ Develop an example Character device driver
- ❑ Enable SPI and I²C sensors drivers and Device Tree nodes
- ❑ Develop device tree overlays for the sensors and enable them automatically and manually from U-boot command line

❑ BitBake and Metadata

- ❑ Learn about Yocto build engine (**BitBake**), its structure, job and history
- ❑ Learn about various BitBake **metadata** types (recipes, classes, conf, patches, ...)
- ❑ Learn how to structure everything in a Yocto **layer**

❑ Recipe Development

- ❑ Learn how to develop recipes for custom applications
- ❑ Learn how to develop recipes for various source types (Git, local, http, ...)
- ❑ Learn how to create and control **package** generation

❑ Images and Distros

- ❑ Learn how **images** are created in Yocto
- ❑ Learn how to control images by adding, removing and optimizing packages
- ❑ Learn about Yocto **distributions** and its features (Init managers, versions, ...)

❑ Lab Summary (*to be communicated later*)

- ❑ Prepare and complete your first Yocto build for the target board
- ❑ Create recipes, images, layers for custom applications
- ❑ Create custom distribution and optimize it to the production build level

❑ Board Support Package

- ❑ Learn how Linux BSP is implemented in Yocto
- ❑ Learn how Linux kernel and bootloaders are implemented
- ❑ Learn how to handle kernel device trees, U-boot configuration, and kernel fragments

❑ Machine Development

- ❑ Learn how to control kernel and bootloader providers and versions
- ❑ Learn how to work with Device Trees and Kernel modules
- ❑ Learn how to work with Yocto OVERRIDES mechanism to support multiple machines and distros
- ❑ Learn about dev and upstream sources and recipe variants extension

❑ Kernel and Initramfs

- ❑ Learn how to develop and integrate initramfs images
- ❑ Learn how to add and handle kernel fragments in Yocto
- ❑ Learn how to optimize the Linux Kernel

❑ Labs Summary (*to be communicated later*)

- ❑ Create custom board-specific machine
- ❑ Develop initramfs image and integrate it in Yocto
- ❑ Develop kernel fragments for the sensors and create machine feature for each

❑ dm-crypt and dm-verity

- ❑ Learn how block and file system encryption levels works
- ❑ Learn how important is data encryption in embedded systems
- ❑ Learn how data integrity feature is done with dm-verity and how it is important
- ❑ Learn how to develop initramfs scripts to implement them

❑ RAUC

- ❑ Learn how A/B Linux update mechanism works
- ❑ Learn all about RAUC OTA solution
- ❑ Learn about squashfs filesystem and adaptive updates
- ❑ Learn how to optimize and handle OTA for production level

❑ Labs Summary (*to be communicated later*)

- ❑ Create data partition with various ways (WIC, ...)
- ❑ Develop encryption and integrity setup in initramfs init process
- ❑ Integrate, develop bundles and test RAUC OTA solution with HTTP docker-based server

What to add to your Resume

-
- ❑ **Operating Systems & Architecture:** Kbuild system, syscalls, Device trees and overlays, ...
 - ❑ **Shell & Scripting:** Bash, POSIX Shell, Shell scripting, ...
 - ❑ **Build Systems & Toolchains:** GCC, binutils, crosstool-ng, cross compilation, canadian cross, make, cmake, ...
 - ❑ **Bootloaders and Secure boot:** U-Boot, dm-crypt, dm-verity, ...
 - ❑ **Kernel Development:** Kernel modules, character devices, GPIO, LED, I²C and SPI subsystems, sysfs, config fragments, ..
 - ❑ **Peripheral Interfaces:** I²C, SPI, GPIO, UART, (i2c-tools, spidev, libgpiod, ...)
 - ❑ **Filesystem & Libraries:** BusyBox, initramfs, static and shared libraries
 - ❑ **Embedded Platforms:** Raspberry Pi Zero 2W, QEMU (ARM)
 - ❑ **Yocto & Build Systems:** BitBake, metadata, machine configs, images, distros, ... RAUC OTA
 - ❑ **Version Control & CI:** Git, git submodules, git patches, ...

+ All the projects that you will work on (*each project with its GitHub repository link*)

Bring freedom to
embedded systems
learning